PRIRUČNIK ZA UČENIKE • ZAŠTITA INFORMACIONIH SISTEMA

MiniSec

Detaljan plan projekta — teorija i zadaci po nedeljama

Dokument 2 od 3: za svaku nedelju prvo teorija koja ti treba, pa detaljan zadatak

Važno: ovde su **teorija i tačni zahtevi**, a ne gotov kod. Rešenja gradiš sam, korak po korak.

Trajanje: 8 nedelja × 2 časa (3 bloka)

Alati: HTML / CSS / JavaScript • Python • CrypTool 2

VODIČ ZA UČENIKE

Kako da koristiš ovaj priručnik

Dokument je podeljen **po nedeljama**. Svaka nedelja ima dva dela: **„Teorija za ovu nedelju"** (pročitaj je pre rada) i **„Zadatak"** (tačno šta treba da napraviš i po čemu znaš da je gotovo). Kako ćeš doći do rešenja — otkrivaš sam, uz pomoć nastavnika i grupe.

Tri alata — tri uloge

Alat	Čemu služi u projektu
HTML / CSS / JavaScript	Izgled i ponašanje aplikacije u pregledaču: forme, dugmad, provera unosa.
Python	„Motor" iza aplikacije: heširanje, šifrovanje, potpisivanje, baza.
CrypTool 2	„Mikroskop": istu operaciju uradiš i tu, pa proveriš da je tvoj rezultat tačan.

Tri bloka projekta

Blok	Nedelje	Šta radimo	Gde se čuvaju podaci
1	1–3	Uvod, okruženje, forme u HTML/CSS/JS	localStorage (u pregledaču)
2	4–6	Python i kriptografija	SQLite (na serveru)
3	7–8	Sustizanje, CrypTool 2, odbrana	SQLite

PRAVILO SAMOSTALNOSTI

AI alati i gotova rešenja sa interneta su **zabranjeni kao prečica**. Smeš da pitaš nastavnika, gledaš zvaničnu dokumentaciju i sarađuješ sa grupom. Na odbrani ćeš morati da **objasniš svaki deo** i da ga **izmeniš uživo** na zahtev. Ko ne razume svoj rad — nije ga ni napravio.

Mali rečnik (vrati se ovamo kad zatreba)

Pojam	Objašnjenje jednom rečenicom
Heš (otisak)	Jednosmerni „otisak prsta" podatka — iz njega se original ne može vratiti.
Salt	Nasumičan dodatak lozinki pre heširanja, da dve iste lozinke ne daju isti otisak.
Šifrovanje (kriptovanje)	Dvosmerno sakrivanje poruke — može se dešifrovati nazad pravim ključem.
Simetrično / asimetrično	Jedan ključ (AES) / dva ključa — javni i privatni (RSA).
Digitalni potpis	Dokaz ko je poslao poruku i da nije menjana.
Sertifikat / PKI	Dokument koji vezuje javni ključ za vlasnika / sistem poverenja oko sertifikata.
Autentifikacija / autorizacija	Provera ko si / određivanje šta smeš da radiš.
localStorage / SQLite	Memorija u pregledaču / prava baza (jedan fajl) na serveru.
Validacija	Provera da je uneti podatak ispravan (npr. da polje nije prazno).

BLOK 1 — Frontend: uvod, okruženje i forme (nedelje 1–3)

U ovom bloku ne radimo Python. Pravimo izgled i ponašanje aplikacije u pregledaču, a podatke čuvamo u **localStorage**.

Nedelja 1 — Uvod i radno okruženje

Teorija za ovu nedelju

Aplikacija koju pravimo radi u **pregledaču**, a grade je tri jezika koji imaju različite uloge:

Jezik	Uloga	Poređenje
HTML	Struktura stranice — šta se nalazi na njoj (naslovi, polja, dugmad).	Skelet
CSS	Izgled — boje, razmaci, raspored, font.	Odeća
JavaScript	Ponašanje — šta se dešava kad klikneš ili kucaš.	Mišići

Kako izgleda struktura HTML stranice

Svaka HTML stranica ima isti osnovni kostur: deklaraciju vrste dokumenta, zaglavlje (*head* — nevidljivi podaci, naslov u kartici, povezivanje sa CSS-om) i telo (*body* — sve što se vidi na stranici). JavaScript se obično povezuje na kraju tela. Ne moraš sad da znaš sintaksu napamet — važno je da razumeš da stranica ima **glavu** (podešavanja) i **telo** (sadržaj).

Zašto razdvajamo fajlove

HTML, CSS i JavaScript držimo u **odvojenim fajlovima** (i folderima) da bi projekat bio pregledan: struktura na jednom mestu, izgled na drugom, ponašanje na trećem. Kad nešto menjaš, tačno znaš gde da tražiš.

> **ŠTA JE „KOSTUR" PROJEKTA**
> Kostur je prazna, ali uredno organizovana struktura foldera i fajlova — kao prazne fioke sa nalepnicama. Prvo napraviš fioke, pa ih tokom nedelja puniš.

Zadatak

Cilj: pripremiti radno okruženje i napraviti kostur projekta sa prvom praznom stranicom.

Korak 1 — Okruženje

- Instaliraj i otvori **Visual Studio Code**.
- Dodaj ekstenziju **Live Server** (preporuka) — omogućava da se stranica automatski osvežava u pregledaču dok radiš.

Korak 2 — Napravi tačno ovakav kostur foldera i fajlova

```
MiniSec/
├── index.html            ← stranica za PRIJAVU (početna)
├── registracija.html     ← stranica za REGISTRACIJU
├── poruke.html           ← stranica za PORUKE (zasad prazna)
│
├── css/
│   └── stil.css          ← izgled svih stranica
│
```

```
└── js/
    ├── prijava.js        ← ponašanje forme za prijavu
    ├── registracija.js   ← ponašanje forme za registraciju
    └── poruke.js         ← ponašanje stranice sa porukama
```

Napomena: foldere `server/` i bazu dodaćemo tek u 4. nedelji (Blok 2). Sada praviš samo ovo iznad.

Fajl / folder	Čemu služi
`index.html`	Prva stranica koja se otvara — forma za prijavu.
`registracija.html`	Forma za pravljenje novog naloga.
`poruke.html`	Stranica sa porukama (gradimo je u 3. nedelji).
`css/stil.css`	Zajednički izgled za sve stranice.
`js/*.js`	Po jedan fajl za ponašanje svake stranice.

Korak 3 — Prva prazna stranica

U `index.html` napravi osnovni kostur stranice koji sadrži:

- naslov u kartici pregledača: **„MiniSec — Prijava"**;
- povezivanje sa `css/stil.css`;
- u telu: veliki naslov **„MiniSec"** i prazan okvir (npr. `div`) u koji ćeš sledeće nedelje staviti formu;
- povezivanje sa `js/prijava.js` na kraju tela.

Otvori stranicu u pregledaču (preko Live Server-a) i proveri da se naslov „MiniSec" vidi.

> **KRITERIJUMI USPEHA**
>
> VS Code radi i stranica se otvara u pregledaču • struktura foldera/fajlova izgleda tačno kao gore • `index.html` prikazuje naslov i povezan je sa CSS i JS fajlom • razumeš čemu služi svaki fajl.

Nedelja 2 — Forma za prijavu i registraciju

Teorija za ovu nedelju

Šta je forma

Forma je deo stranice u koji korisnik **unosi podatke**. Sastoji se od **polja za unos** (npr. tekst, lozinka), **dugmadi** (npr. „Prijava") i poruka (npr. greška). Polje za lozinku prikazuje **tačkice** umesto znakova da niko ne vidi šta kucaš.

localStorage — gde pregledač pamti podatke

U Bloku 1 podatke čuvamo u **localStorage** — maloj memoriji ugrađenoj u pregledač. Ona čuva podatke kao parove „naziv → vrednost" i oni **ostaju** i kad osvežiš stranicu. U njega ćemo upisivati korisnike (a kasnije i poruke). Isti localStorage vide sve stranice istog sajta, pa podatak upisan na registraciji možeš pročitati na prijavi.

Kvalitet lozinke — šta jaka lozinka mora da sadrži

Jačina lozinke zavisi od toga koliko je teško pogoditi. Dve stvari su ključne: **dužina** (najvažnija) i **raznovrsnost znakova**. U našem projektu lozinka je prihvatljiva ako ispunjava sledeće uslove:

Uslov	Zahtev	Primer
Dužina	najmanje **8** znakova (preporuka 10+)	8 i više
Malo slovo	bar jedno **a–z**	a, m, z
Veliko slovo	bar jedno **A–Z**	A, M, Z
Broj	bar jedna **cifra 0–9**	3, 7
Specijalan znak	bar jedan **! ? @ # $ % & ***	@, #, !

Primer slabe lozinke: `sunce` (kratka, samo mala slova). Primer jake: `Sunce@Leto7` (duga, ima sve vrste znakova). **Savet:** duža fraza koju lako pamtiš često je jača i praktičnija od kratke zbrke znakova.

Jačina lozinke — tri nivoa (za merač)

Nivo	Kada	Boja trake
Slaba	kraća od 8, ili ispunjava samo 1–2 uslova	crvena
Srednja	8+ znakova i ispunjava 3 uslova	narandžasta
Jaka	10+ znakova i ispunjava sva 4 uslova raznovrsnosti	zelena

Validacija — šta se dešava kad nešto nije u redu

Validacija znači provera unosa pre nego što se podatak prihvati. Korisniku uvek treba jasno reći šta nije u redu. Pravila ponašanja koja tvoja forma mora da poštuje:

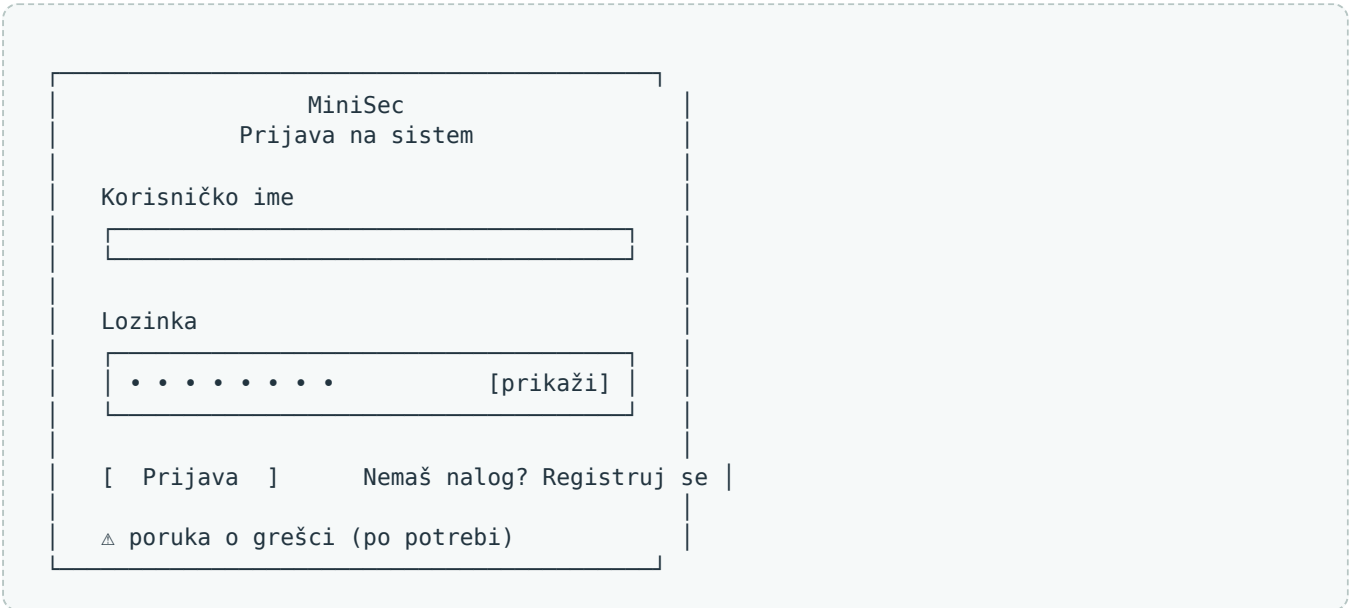
Situacija	Šta forma treba da uradi
Polje nije popunjeno	Označi polje (npr. crveni okvir), ispiši poruku „Ovo polje je obavezno" i **ne šalji** formu.
Lozinka ne ispunjava uslove (registracija)	Ispiši **koji** uslov nedostaje (npr. „Lozinka mora imati veliko slovo") i blokiraj registraciju.
Lozinka i potvrda se ne poklapaju	Poruka „Lozinke se ne poklapaju".
Korisničko ime već postoji	Poruka „Korisnik sa tim imenom već postoji".
Uspešna registracija	Poruka o uspehu i prelazak na stranicu za prijavu.
Pogrešna prijava	Poruka „Pogrešno korisničko ime ili lozinka" — **ne otkrivaj** šta je tačno pogrešno.
Uspešna prijava	Zapamti ko je prijavljen i pređi na stranicu sa porukama.

> **ZAŠTO KOD PRIJAVE NE KAŽEMO ŠTA JE TAČNO POGREŠNO**
>
> Ako bismo rekli „korisnik ne postoji", napadač bi saznao koja imena postoje, a koja ne. Zato je poruka uvek ista i neutralna — to je mali, ali važan bezbednosni detalj.

Zadatak

Cilj: napraviti forme za prijavu i registraciju koje rade u pregledaču, sa proverom unosa i meračem jačine lozinke.

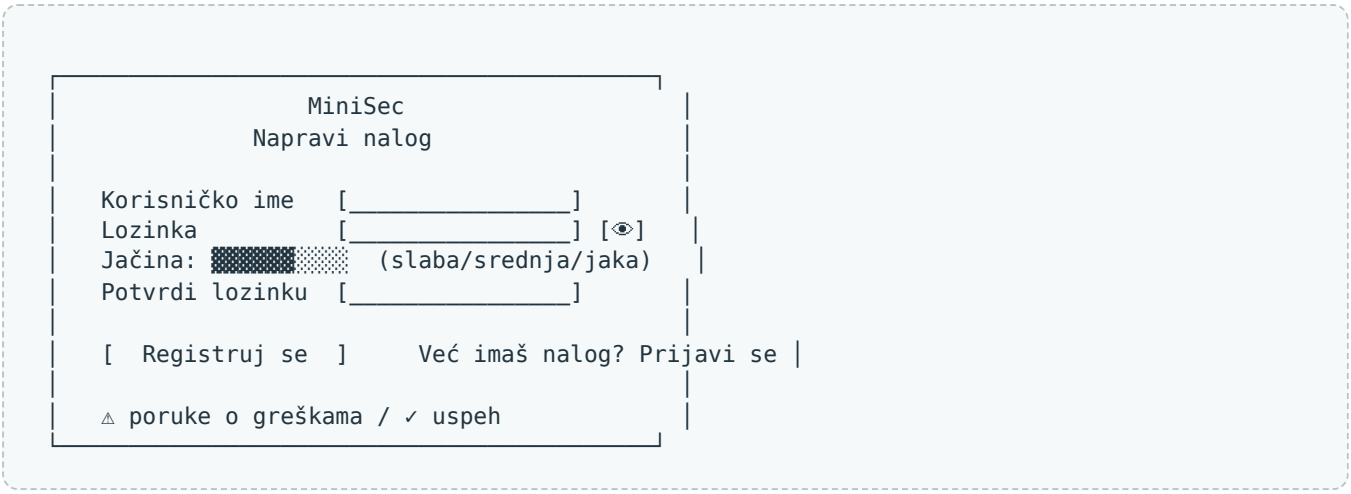
Forma za PRIJAVU (index.html) — mora da ima



- Polje **Korisničko ime** (običan tekst).
- Polje **Lozinka** (prikazuje tačkice) sa opcijom **prikaži/sakrij**.
- Dugme **Prijava** i link ka **Registraciji**.

- Prostor za **poruku o grešci**.

Forma za REGISTRACIJU (registracija.html) — mora da ima



- Polja: **Korisničko ime**, **Lozinka**, **Potvrdi lozinku**.
- **Indikator jačine** ispod polja za lozinku (traka koja uživo menja boju i tekst: slaba/srednja/jaka po pravilima iz teorije).
- Dugme **Registruj se** i link ka **Prijavi**.
- Prostor za poruke (greške i uspeh).

Funkcionalnost koju treba postići

- **Merač jačine** se menja **uživo** dok korisnik kuca lozinku.
- **Registracija:** proveri sve uslove (popunjenost, uslovi lozinke, poklapanje potvrde, da ime ne postoji); ako je sve u redu, upiši korisnika u **localStorage** i pređi na prijavu.
- **Prijava:** pronađi korisnika u localStorage i uporedi lozinku; na uspeh zapamti ko je prijavljen i pređi na `poruke.html`; na neuspeh ispiši neutralnu poruku.
- Sva pravila iz tabele **„Validacija"** moraju da važe.

> **KRITERIJUMI USPEHA**
>
> Registracija upisuje korisnika u localStorage i poštuje sve uslove lozinke • merač jačine radi uživo • prijava radi i vodi na stranicu poruka • prazna polja i greške daju jasnu poruku i blokiraju slanje • forma je pregledna.

Nedelja 3 — Forma za poruke i kraj Faze 1

Teorija za ovu nedelju

Kako stranice dele podatke

Sve stranice istog sajta dele **isti localStorage**. Zato korisnici koje si upisao na registraciji možeš pročitati i na stranici poruka. Takođe treba negde da zapamtimo **ko je trenutno prijavljen** — to je mala informacija koju upišemo pri prijavi, a obrišemo pri odjavi (tzv. sesija prijavljenog korisnika).

Zašto localStorage nije bezbedan

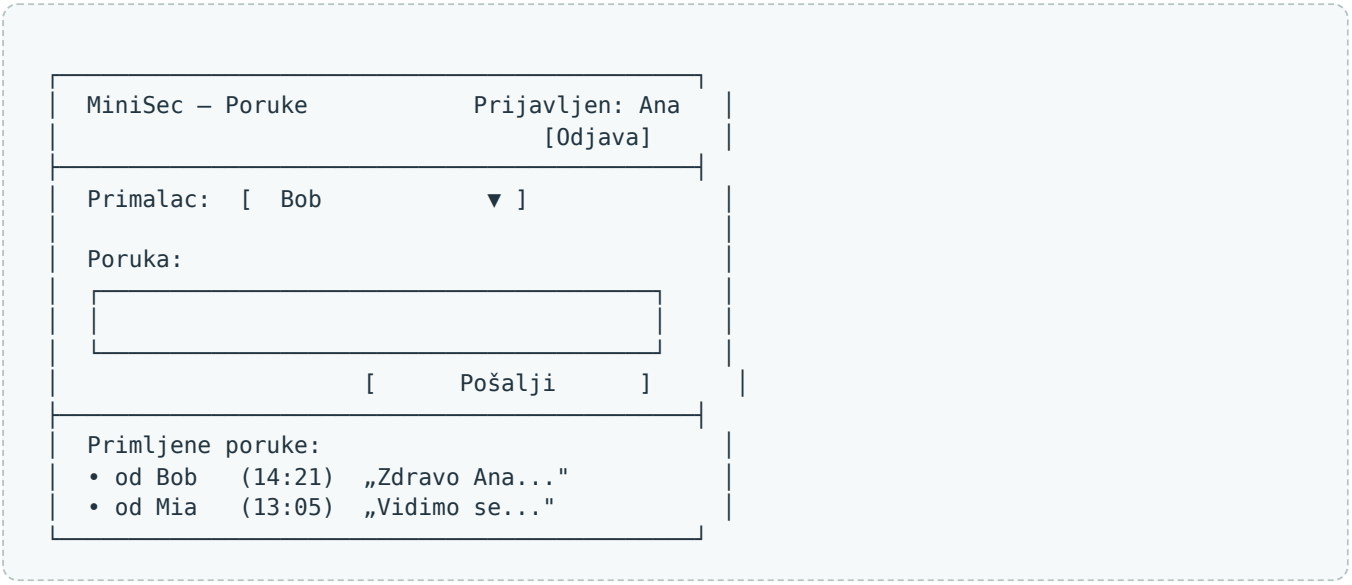
localStorage je **vidljiv i izmenjiv** svakome ko otvori alatke pregledača. To znači da bi prava lozinka tu stajala „na dlanu". Zato ga koristimo **samo kao privremeni prototip** dok forme ne prorade. U Bloku 2 ćemo zato preseliti podatke na server, gde ih korisnik ne vidi.

> **OVO JE GLAVNI „AHA" MOMENAT BLOKA 1**
>
> Na kraju ove nedelje svojim očima ćeš videti svoje podatke u pregledaču. Upravo to je razlog zašto u 4. nedelji sve selimo na server i bazu.

Zadatak

Cilj: napraviti stranicu sa porukama, spojiti sve u celinu i uočiti granice localStorage-a.

Forma za PORUKE (poruke.html) — mora da ima



- Zaglavlje: oznaka **ko je prijavljen** i dugme **Odjava**.
- **Primalac** — padajući spisak postojećih korisnika.
- **Tekst poruke** — veće polje za unos.
- Dugme **Pošalji**.
- **Lista primljenih poruka** (pošiljalac, vreme, tekst).

Funkcionalnost koju treba postići

Situacija	Šta treba da se desi
Stranicu otvara neprijavljeni korisnik	Vrati ga na stranicu za prijavu (ne sme da vidi poruke).
Slanje poruke	Upiši u localStorage: pošiljalac (trenutni korisnik), primalac, tekst, vreme.
Prazan tekst ili nije izabran primalac	Poruka „Izaberi primaoca i unesi tekst"; ne šalji.
Prikaz primljenih	Prikaži samo poruke čiji je primalac trenutni korisnik.
Odjava	Obriši podatak o prijavljenom korisniku i vrati na prijavu.

Korak na kraju nedelje — pogledaj svoje podatke

- Otvori alatke pregledača (taster **F12**) → kartica **Application** → **Local Storage**.
- Pronađi upisane **korisnike** i **poruke**. Zapiši šta sve vidiš (da li se vide lozinke? ko bi još ovo mogao da vidi?).

> **KRITERIJUMI USPEHA**
>
> Stranica poruka radi samo za prijavljenog korisnika • slanje i prikaz poruka rade preko localStorage • odjava radi • prazan unos daje poruku • umeš da objasniš **zašto localStorage nije bezbedan**.

BLOK 2 — Python i kriptografija (nedelje 4–6)

Sada uvodimo **Python**. Obrada lozinki i čuvanje podataka sele se na server, a baza **SQLite** zamenjuje localStorage.

Nedelja 4 — Python, heširanje i prelaz na SQLite

Teorija za ovu nedelju

Heš funkcija

Heš funkcija pravi „otisak" teksta — niz znakova uvek iste dužine. Osobine: **jednosmerna** (iz otiska se ne vraća original), **osetljiva** (promena jednog slova menja ceo otisak — „avalanche"), **deterministička** (isti ulaz daje isti otisak). Koristimo **SHA-256**.

> **LOZINKE SE HEŠUJU, NE KRIPTUJU**
>
> Sajt čuva samo **otisak** lozinke. Pri prijavi sistem **ponovo napravi otisak** unetog i uporedi dva otiska — ništa se ne „dešifruje". Tako ni onaj ko ukrade bazu ne vidi prave lozinke.

Salt

Ako dvoje ljudi ima istu lozinku, otisci bi bez dodatka bili identični. Zato svakoj lozinci dodamo **salt** — kratak nasumičan niz, različit za svakog — pa iste lozinke daju **različite** otiske. Salt se čuva uz otisak (nije tajna).

Napadi na lozinke

Napad	Kako radi	Odbrana
Rečnički	Proba reči iz spiska čestih lozinki.	Ne koristiti česte reči; duže fraze.
Gruba sila	Proba sve kombinacije redom.	Dužina lozinke (svaki znak višestruko uvećava posao).

Baza i tabela; SQLite

Baza čuva podatke uredno, u **tabelama** (slično Excel-u: kolone i redovi). **SQLite** je prava baza koja je ceo jedan fajl i ugrađena je u Python. Napravićemo dve tabele:

Tabela	Kolone
korisnici	korisničko ime • otisak lozinke (sa salt-om) • uloga • (kasnije) javni ključ
poruke	pošiljalac • primalac • tekst/šifrat • (kasnije) potpis • vreme

Zadatak

Cilj: bezbedno čuvanje lozinke u Pythonu i selidba podataka iz localStorage u SQLite.

- Napravi Python program (terminalski) koji od lozinke pravi **heš sa salt**-om i ume da **proveri** lozinku upoređivanjem otisaka. Pokaži da ista lozinka dva puta daje **različite** otiske.
- Uporedi **čist** SHA-256 otisak (bez salt-a) sa rezultatom u **CrypTool 2** — moraju biti identični.
- Napravi program koji **rečnički** i **grubom silom** pokušava da pogodi jednu **slabu** lozinku i **meri vreme**. Isprobaj kako vreme raste kad lozinku produžiš.
- Napravi **SQLite** bazu sa tabelom `korisnici`; **ponovo registruj** par naloga tako da se sada upisuju u bazu (a ne u localStorage).

OKVIR ZA NAPADE

Napad radiš **samo** na otisak koji si sam napravio, lokalno — to je **odbrambena** vežba koja pokazuje zašto su kratke lozinke loše. Ništa se ne radi na tuđim ili stvarnim sistemima.

KRITERIJUMI USPEHA

Heš sa salt-om radi i provera lozinke prolazi • čist heš se poklapa sa CrypTool 2 • napad pokazuje razliku slabe i jake lozinke • korisnici se sada čuvaju u SQLite bazi, a ne u pregledaču.

Nedelja 5 — Šifrovanje poruka: AES i RSA

Teorija za ovu nedelju

Za razliku od lozinki (koje heširamo), **poruke šifrujemo** — jer primalac mora da pročita original. Dva pristupa:

Simetrično — AES (jedan ključ)

Isti ključ i zaključava i otključava poruku. Brzo, ali postoji problem: kako obe strane da dobiju isti tajni ključ a da ga niko ne presretne?

Asimetrično — RSA (dva ključa)

Svako ima **par**: **javni** ključ (deli se svima) i **privatni** (čuva se u tajnosti). Što se zaključa javnim, otključava se samo odgovarajućim privatnim. Zato ti bilo ko može poslati tajnu poruku tvojim javnim ključem, a samo je ti čitaš svojim privatnim.

> **ODAKLE RSA KLJUČEVI**
>
> RSA koristi dva velika prosta broja. U CrypTool 2 videćeš oznake: `p` i `q` (prosti brojevi), `n` (njihov proizvod), `e` (javni deo), `d` (privatni deo). Mali ključevi su nebezbedni jer se lako „razbiju".

Zadatak

Cilj: šifrovati i dešifrovati poruku, prvo jednim pa sa dva ključa.

- Program koji **AES**-om šifruje poruku i vraća je u original; proveri isti rezultat u CrypTool 2.
- Generiši **RSA** par ključeva; šifruj javnim, dešifruj privatnim.
- U CrypTool 2 pronađi i objasni vrednosti `p, q, n, e, d`.
- Šifrovane poruke upiši u **SQLite** tabelu `poruke`.

> **KRITERIJUMI USPEHA**
>
> Poruka se šifruje i vraća u original • umeš da objasniš razliku simetrično/asimetrično i zašto se javni ključ sme deliti • poruke se čuvaju u bazi.

Nedelja 6 — Razmena ključeva, potpis i prava pristupa

Teorija za ovu nedelju

Diffie-Hellman — tajna bez slanja tajne

Dve strane (Alisa i Bob) javno se dogovore o početnim vrednostima, svaka doda svoju **tajnu** i razmene samo **međurezultate**. Kombinovanjem tuđeg međurezultata sa svojom tajnom, obe dobiju **isti** konačni ključ. Prisluškivač (Eva) vidi javne vrednosti, ali bez ičije tajne ne može da izračuna ključ.

Digitalni potpis

Pošiljalac „zaključa" otisak poruke **svojim privatnim** ključem; svako proverava **njegovim javnim**. Ako provera prođe: poruka je stvarno od njega (**autentičnost**) i nije menjana (**celovitost** — najmanja izmena ruši otisak).

Sertifikat i PKI

Kako znam da je neki javni ključ stvarno Bobov? **Digitalni sertifikat** vezuje identitet za javni ključ i potpisan je od tela kome se veruje. **PKI** je ceo sistem izdavanja i provere sertifikata (zato veruješ sajtu sa katancem).

Autentifikacija vs autorizacija; ACL

Autentifikacija = „dokaži ko si" (lozinka). **Autorizacija** = „šta smeš da radiš". Za autorizaciju koristimo **uloge** (npr. korisnik, administrator) i **listu kontrole pristupa (ACL)** — ko sme šta.

Zadatak

Cilj: razmena ključa, potpis, sertifikat i kontrola pristupa.

- Simulacija **Alisa–Bob** koja daje isti zajednički ključ; pokaži da Eva ne može da ga izračuna.
- **Potpiši** poruku privatnim ključem i proveri javnim; zatim **izmeni jedno slovo** i pokaži da provera **pada**.
- Napravi jednostavan **sertifikat** i pročitaj mu sadržaj (vlasnik, javni ključ, izdavalac, rok).
- Uvedi bar dve **uloge** i **ACL**; pokaži da korisnik biva **odbijen** kad pokuša radnju koja mu nije dozvoljena.

> **KRITERIJUMI USPEHA**
>
> Alisa i Bob dobiju isti ključ • potpis pada za izmenjenu poruku • umeš da pročitaš sertifikat • ACL stvarno ograničava radnje • razlikuješ autentifikaciju od autorizacije.

BLOK 3 — Sustizanje, CrypTool 2 i odbrana (nedelje 7–8)

Nedelja 7 — Sustizanje i integracija

Teorija za ovu nedelju

Bez novog gradiva. **Integracija** znači spajanje svih delova u jednu celinu koja radi od početka do kraja. **Testiranje** znači namerno probati i „loše" slučajeve da vidimo kako se aplikacija ponaša — to je deo bezbednosti.

Zadatak

Cilj: dovršiti sve zaostalo i spojiti aplikaciju u celinu.

- **Dovrši** module koji su ostali nedovršeni.
- **Poveži** sve u jedan tok: prijava → šifrovana poruka → potpis → prava pristupa, uz SQLite u pozadini.
- **Testiraj** sledeće „šta ako" slučajeve i zapiši ponašanje:

Test	Očekivano ponašanje
Pogrešna lozinka	Neutralna poruka o grešci, bez pada aplikacije.
Prazno polje	Poruka „obavezno polje", slanje blokirano.
Izmenjena poruka pre provere potpisa	Provera potpisa **pada**.
Zabranjena radnja (ACL)	Korisnik biva **odbijen**.

KRITERIJUMI USPEHA

Aplikacija radi od početka do kraja • svi testovi iz tabele daju očekivano ponašanje.

Nedelja 8 — CrypTool 2 simulacija i odbrana

Teorija za ovu nedelju

CrypTool 2 i tvoja aplikacija rade po **istom standardu** (SHA-256, AES, RSA…), pa za isti ulaz daju **isti rezultat**. Zato CrypTool 2 služi kao **provera** — kao kad račun uradiš u glavi, pa proveriš digitronom. Na odbrani pokazuješ da razumeš **zašto** nešto radi, ne samo da radi.

Zadatak

Cilj: kratko proveriti module u CrypTool 2, pa predstaviti i odbraniti rad.

- **Kratka simulacija u CrypTool 2:** provuci iste ulaze kroz **heš, AES, RSA** i **Diffie-Hellman** šablone i pokaži poklapanje sa aplikacijom; napravi **snimke ekrana** kao dokaz.
- Pripremi kratku **dokumentaciju** (1–2 strane): šta radi, kako se pokreće, koje teme pokriva.
- Pripremi **prezentaciju** (5–10 min) i spremi se za **pitanja** i **izmenu uživo**.

KRITERIJUMI USPEHA

Rezultati u CrypTool 2 se poklapaju • jasno izlažeš • objasniš svaki deo svojim rečima • uspešno izmeniš deo na zahtev • odgovoriš na mini-kviz.

ZAPAMTI ZA ODBRANU

Izmena uživo nije zamka — to je tvoja prilika da pokažeš da je rad **tvoj**. Ako razumeš šta si napravio, ta promena ti je laka.

Šta se predaje na kraju

- **Radna aplikacija** (forme + Python deo) sa modulima tvoje grupe.
- **Kratka dokumentacija** (1–2 strane).
- **Dokazi iz CrypTool 2** (snimci ekrana sa poklapanjem).
- **Prezentacija** (5–10 min) sa odbranom — očekuj pitanja i izmenu uživo.